

Scenario 1: File Upload

Weak Prompt: Generate test cases for file upload.

ChatGPT Response:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result
TC_001	Upload a valid file	1. Open upload page 2. Select a valid file 3. Click Upload	document.pdf (allowed type, within size limit)	File uploads successfully
TC_002	Upload multiple valid files	1. Select multiple files 2. Click Upload	3 valid files	All files upload successfully
TC_003	Upload file using drag and drop	1. Drag file into upload area 2. Drop file	Valid file	File is uploaded successfully
TC_004	Cancel file selection before upload	1. Select a file 2. Click cancel/remove	Any file	File is removed and upload is canceled
TC_005	Replace selected file	1. Select file A 2. Select file B	fileA.pdf, fileB.pdf	File B replaces file A
TC_006	Upload without selecting a file	1. Open upload page 2. Click Upload	No file	Validation message displayed
TC_007	Upload file with unsupported extension	1. Select unsupported file 2. Upload	.exe, .bat	Upload rejected with proper error message
TC_008	Upload empty file	1. Create empty file 2. Upload	0 KB file	Upload rejected or handled based on requirements
TC_009	Upload file exceeding maximum size	1. Select large file 2. Upload	File > configured size limit	Upload fails with size error
TC_010	Upload file at maximum allowed size	1. Select file equal to limit 2. Upload	File = max size	Upload succeeds
TC_011	Upload file just below maximum size	Upload file slightly smaller than limit	Valid large file	Upload succeeds
TC_012	Upload file with special characters in filename	Upload file named test@#\$%.pdf	Special-character filename	File handled correctly
TC_013	Upload file with very long filename	Upload file with long name	Filename > allowed length	Proper validation displayed

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result
TC_014	Upload duplicate file	Upload same file twice	Same filename/content	System prevents duplicates or creates versions as expected
TC_015	Upload file with uppercase extension	Upload .PDF instead of .pdf	REPORT.PDF	File accepted if extension is case-insensitive
TC_016	Upload file with renamed extension	Rename .exe to .pdf and upload	Fake PDF file	File validation detects invalid content
TC_017	Upload corrupted file	Upload damaged file	Corrupt PDF/image	Upload rejected or error shown
TC_018	Upload file after session timeout	Login, wait for timeout, upload file	Valid file	User redirected or asked to re-authenticate
TC_019	Refresh page during upload	Start upload and refresh	Large file	Upload behavior matches requirement
TC_020	Close browser during upload	Start upload and close browser	Large file	Upload stops safely
TC_021	Network interruption during upload	Disconnect network during upload	Valid file	Error shown and upload fails gracefully
TC_022	Resume interrupted upload	Interrupt and restart upload	Large file	Upload resumes if supported
TC_023	Verify upload progress indicator	Upload large file	Large file	Progress bar/status updates correctly
TC_024	Verify uploaded file name	Upload file	test.pdf	Displayed filename matches original
TC_025	Verify uploaded file download	Upload then download file	Valid file	Downloaded file matches uploaded file
TC_026	Verify file storage location	Upload file and check backend	Valid file	File stored correctly
TC_027	Upload same file from different users	User A uploads file, User B uploads same file	Same file	Access behavior follows permissions
TC_028	Verify permission restrictions	Upload without required permission	Restricted user	Upload denied
TC_029	Upload malicious file	Upload malware test file	EICAR test file	File blocked/quarantined
TC_030	Verify virus scanning	Upload infected file	Virus sample	System detects and blocks file
TC_031	Verify authentication requirement	Access upload URL without login	Unauthenticated user	Redirected to login

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result
TC_032	Verify HTTPS upload	Upload through application	Valid file	File transferred securely
TC_033	Test concurrent uploads	Upload multiple files simultaneously	Multiple large files	System handles uploads correctly
TC_034	Test upload performance	Upload large file under normal load	500MB file	Upload completes within expected time
TC_035	Test upload under high traffic	Multiple users upload files	Many concurrent users	System remains stable
TC_036	Verify error messages	Trigger upload failures	Invalid files	Clear, user-friendly errors displayed
TC_037	Verify file type filtering	Open file picker	Various file types	Only allowed types selectable
TC_038	Verify mobile upload	Upload from mobile device	Image/document	Upload works correctly
TC_039	Verify browser compatibility	Test Chrome, Firefox, Edge, Safari	Valid file	Upload works across supported browsers
TC_040	Verify accessibility	Navigate upload using keyboard/screen reader	Valid file	Upload control is accessible

ICEPOT Prompt:

Context

Act as a Functional tester – Senior level and write test cases to test “File Upload” functionality in an insurance page, which will be displayed after the login page. 70% of weightage should be given to functional test cases and remaining 30% to security test cases. Count of test cases should be 10-12.

Instruction

[MANDATORY] Generate testcases for validating the functionality with supported file types – JPEG and PNG

[MANDATORY] Generate testcases for validating the functionality with maximum size limit of 5 MB

[MANDATORY] Generate testcases for validating the functionality with success toast message after uploading the file successfully

[MANDATORY] Generate testcases for validating the functionality with failure toast message when there is error during the upload

[CRITICAL] Include testcases for validating the security aspects – The UI should prevent the file from fully uploading (e.g., stopping at 99%)

[HIGH PRIORITY] Generate test cases to verify a clear, user-friendly error is displayed (e.g., "The file could not be processed. Please upload a different file."). Error message should not include technical jargon or revealing that a virus detected, as this can give hackers insight into security attack

[LOW PRIORITY] Generate testcases for validating the file name length and typography

[LOW PRIORITY] Generate testcases for validating the styling and alignment of page

[HIGH PRIORITY] Generate test case for validating the file upload using drag and drop

[HIGH PRIORITY] Generate test case for validating the file upload by replacing the already uploaded file

[STRICT] Generate test case for validating the functionality through chrome browser and windows OS

Example:

TCID | Description | Priority | Preconditions | Steps | Expected Result

Persona

Act as a Peer to review the testcases

Output

Downloadable excel file

Tone:

Generate the test cases in a way it should be presented to Business/ Higher management for demonstration purposes

ChatGPT Response:

I have created the Excel file containing **12 senior-level functional and security test cases** for the **Insurance File Upload functionality**.

The test cases are structured for **Business / Higher Management demonstration** and include:

-  70% focus on functional validation
-  30% focus on security validation
-  Supported file types: JPEG and PNG
-  Maximum file size validation: 5 MB
-  Success toast message validation
-  Failure toast message validation
-  Security validation for upload prevention (e.g., stopping before completion)
-  User-friendly security error message validation
-  Drag-and-drop upload validation
-  Replace existing uploaded file validation
-  Filename length and typography validation
-  Page styling/alignment validation
-  Chrome browser + Windows OS validation

You can download the Excel file here:

Downloaded File Screenshot:

Insurance_File_Upload_Test_Cases						
A1	TCID					
1	TCID	Description	Priority	Category	Preconditions	Expected Result
2	TC_001	Verify user can upload a supported JPEG file from insurance page	High	Functional	User is logged in through Chrome browser on Windows OS and is on insurance page file upload section	JPEG file uploads successfully and success toast message is displayed confirming successful upload.
3	TC_002	Verify user can upload a supported PNG file from insurance page	High	Functional	User is logged in and file upload section is available	PNG file uploads successfully and success toast message is displayed.
4	TC_003	Verify file size validation for files exceeding maximum limit of 5 MB	High	Functional	User is logged in and upload control is available	Upload is blocked and user receives a clear validation message indicating maximum allowed file size is 5 MB.
5	TC_004	Verify failure toast message is displayed when upload encounters an error	High	Functional	User is logged in and upload page is displayed	Upload fails gracefully and failure toast message is displayed without losing page functionality.
6	TC_005	Verify successful file upload using drag and drop functionality	High	Functional	User is logged in and drag-drop upload area is visible	File is uploaded successfully and success toast message is displayed.
7	TC_006	Verify user can replace an already uploaded file	High	Functional	A valid JPEG/PNG file has already been uploaded	Previous file is replaced with new file and success toast message confirms the latest upload.
8	TC_007	Verify clear user-friendly error message is displayed for invalid or unsafe file processing	High	Security	User is logged in and upload section is available	Upload is rejected and message displayed is user-friendly, such as 'The file could not be processed. Please upload a different file.' Message must not expose technical details or security detection information.
9	TC_008	Verify UI prevents suspicious file upload from completing fully	Critical	Security	User is logged in and upload functionality is available	UI prevents file from fully uploading (for example upload stops before completion at 99%) and prevents file availability after upload attempt.
10	TC_009	Verify unsupported file types are rejected	High	Functional	User is logged in	File upload is blocked and appropriate validation message is displayed.
11	TC_010	Verify file name length and typography validation	Low	Functional	User is logged in	System handles file naming rules correctly and displays a clear message if filename exceeds allowed length or contains unsupported characters.
12	TC_011	Verify page styling and alignment of file upload section	Low	Functional	User is logged in and insurance page is loaded	Upload button, labels, messages, and controls are properly aligned and consistent with application design standards.
13	TC_012	Verify upload functionality works correctly in Chrome browser on Windows OS	High	Functional	Windows OS machine with Chrome browser available and valid user credentials	Upload works successfully in Chrome on Windows OS with correct success toast notification.

Observation:

- Given 10-12 as the test case count, the test cases were generated with the maximum number in the range (12 test cases).
- Even though the Category column was not specifically mentioned in the Examples section, based on other requirements, the Category column was added to the generated Excel file.
- When the requirement/user need is precisely given, it makes the job easier for the AI model and reduces repeated prompting/user input.

Scenario 2: Shopping Cart

Weak Prompt: Test the cart functionality.

ChatGPT Response:

I'll approach this as a Senior Functional Tester and provide test cases for an e-commerce Cart functionality. The focus is primarily functional testing, with some security coverage.

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Type
TC_CART_001	Add a product to cart	1. Login to application2. Navigate to product page3. Click "Add to Cart"	Valid product	Product should be added successfully and cart count should update	Functional

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Type
TC_CART_002	Verify cart details	1. Add product to cart 2. Open cart page	Product with name, price, quantity	Cart should display correct product details, price, image, and quantity	Functional
TC_CART_003	Add multiple products to cart	1. Add different products 2. Navigate to cart	Product A, Product B	All selected products should appear in cart	Functional
TC_CART_004	Update product quantity in cart	1. Add product 2. Change quantity using +/– option or input field	Quantity: 2	Cart quantity should update and total price should recalculate correctly	Functional
TC_CART_005	Remove product from cart	1. Add product 2. Click Remove/Delete option	Existing cart item	Product should be removed and cart total should update	Functional
TC_CART_006	Verify cart persistence after logout/login	1. Add items to cart 2. Logout 3. Login again	Registered user account	Cart items should persist according to application requirements	Functional
TC_CART_007	Verify cart behavior for guest user	1. Browse without login 2. Add product to cart 3. Close and reopen browser	Guest user	Cart should behave according to guest cart policy (retain or clear items)	Functional
TC_CART_008	Verify maximum quantity limit	1. Add product 2. Increase quantity beyond allowed limit	Quantity exceeding stock	System should prevent invalid quantity and show appropriate message	Functional
TC_CART_009	Verify out-of-stock product handling	1. Add product with limited inventory 2. Change quantity	Stock = 0 or limited	User should receive stock availability message and	Functional

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Type
		beyond available stock		checkout should be restricted	
TC_CART_010	Verify cart total calculation	1. Add multiple products 2. Apply quantity changes 3. Check total	Multiple products with different prices	Subtotal, tax, discounts, and final total should calculate correctly	Functional
TC_CART_011	Validate unauthorized cart access	1. Copy cart URL 2. Open in another browser/session without authentication	Invalid session	Application should enforce access rules based on design	Security
TC_CART_012	Validate cart input security	1. Modify quantity parameter/request payload 2. Send malicious input	Script/SQL payload	Application should reject malicious input and sanitize data	Security

ICEPOT Prompt:

Context

Act as an end user and write test cases to test the "Shopping Cart" functionality of a retail application. The component is to be manually validated for functionality. Count of test cases should be 12-15.

Instruction

[CRITICAL] Generate test cases to validate that adding an item to the shopping cart is allowed for guest, newly signed-up, and signed-in users.

[MANDATORY] Generate test cases to validate the functionality for updating the shopping cart by editing item quantities.

[MANDATORY] Generate test cases to validate the functionality for updating the shopping cart by adding more items.

[MANDATORY] Generate test cases to validate the functionality for deleting items from the shopping cart.

[MANDATORY] Generate test cases to validate that the shopping cart displays the consolidated price total of all available items.

[HIGH PRIORITY] Generate test cases to validate that the shopping cart's consolidated price total excludes tax calculations.

[HIGH PRIORITY] Generate test cases to validate that the shopping cart's consolidated count is updated based on the Add/Update/Delete scenarios.

[MANDATORY] Generate test cases to validate the shopping cart's discount/coupon section by providing valid and invalid (expired/already used) coupons.

[HIGH PRIORITY] Generate test cases to validate the shopping cart by applying multiple discount coupons. It should allow the user to utilize more than one valid coupon at a time.

[MEDIUM PRIORITY] Generate test cases to validate the shopping cart display order. The most recently added items should be displayed at the top.

[LOW PRIORITY] Generate test cases to validate the styling and alignment of the page.

[HIGH PRIORITY] Generate test cases to validate the shopping cart's ability to "Save for Later" any added items.

[MANDATORY] Generate a test case to validate the shopping cart when it contains "Out of Stock" items. A message should appear below the item in a red font stating "Item went out of stock," and updating the quantity should be disabled.

[MANDATORY] Generate a test case to validate that a user cannot place an order if there are "Out of Stock" items in the cart.

[STRICT] Generate test cases to validate functionality across the Chrome browser on Windows OS, as well as on the iOS mobile app.

Example:

TCID | Description | Priority | Preconditions | Steps | Expected Result

Persona

Act as a Business Person to review the testcases

Output

Downloadable excel file. "All Border" to be applied for the content within generated file.

Tone:

Generate the test cases so they are reusable for similar applications.

ChatGPT Response:

I have created the reusable Shopping Cart functional test cases workbook in Excel format. The file includes:

- 14 test cases covering all requested scenarios
- Priority classification
- Preconditions
- Manual execution steps
- Expected results
- Coverage for:
 - Guest, newly signed-up, and signed-in users
 - Add/update/delete cart operations
 - Quantity changes
 - Price total and tax exclusion
 - Cart count updates
 - Coupon validation (valid, expired, used)
 - Multiple coupon application
 - Display order
 - Save for Later
 - Out-of-stock handling
 - Order prevention for out-of-stock items
 - Chrome on Windows and iOS mobile app validation
 - UI styling/alignment checks
- All cells have borders applied as requested.

Download the Excel file here:

Shopping_Cart_Functional_Test_Cases										
ID	Description	Priority	Prerequisites	Steps	Expected Result					
TC_001	Validate adding items to shopping cart for guest user	Critical	User is on retail application without login; product is available	1. Browse products as guest user 2. Select an item 3. Click Add to Cart 4. Open cart	Item is added successfully and cart displays the selected item for guest user.					
TC_002	Validate adding items to shopping cart for newly signed-up user	Critical	New user account is created successfully; product is available	1. Sign up as a new user 2. Login 3. Add product to cart 4. Open cart	Item is added successfully and associated with the newly created account.					
TC_003	Validate adding items to shopping cart for signed-in user	Critical	Existing user is logged in; product is available	1. Add one or more products to cart 2. Navigate to shopping cart	Selected products are displayed in the cart for the signed-in user.					
TC_004	Validate updating shopping cart quantity by editing item quantity	Mandatory	Cart contains an available item with quantity 1	1. Open cart 2. Change item quantity to a valid higher/lower value 3. Save/update cart	Item quantity updates successfully and price/count values refresh accordingly.					
TC_005	Validate updating shopping cart by adding more items	Mandatory	Cart already contains at least one item	1. Continue shopping 2. Add another item 3. Open cart	New item is added while existing cart items remain unchanged.					
TC_006	Validate deleting items from shopping cart	Mandatory	Cart contains multiple items	1. Open cart 2. Click delete/remove for an item	Selected item is removed and cart details update correctly.					
TC_007	Validate consolidated price total and tax exclusion in shopping cart	High	Cart contains multiple items with known prices and applicable taxes configured	1. Add multiple items 2. Delete an item 3. Compare displayed total with item price sum excluding tax	Cart displays consolidated item price total only and excludes tax calculations.					
TC_008	Validate consolidated item count updates for Add/Update/Delete scenarios	High	Cart contains items	1. Add an item 2. Increase quantity 3. Delete an item 4. Observe item count	Cart count updates correctly after every add, quantity update, and delete action.					
TC_009	Validate coupon functionality with valid and invalid coupons	Mandatory	Cart has eligible items; valid, expired, and used coupons are available	1. Enter valid coupon 2. Apply expired coupon 3. Apply already used coupon	Valid coupon applies successfully. Expired and already used coupons are rejected with appropriate messages.					
TC_010	Validate applying multiple valid discount coupons	High	Multiple valid coupons are available and cart meets conditions	1. Add eligible items 2. Apply first valid coupon 3. Apply second valid coupon	System allows multiple valid coupons to be applied simultaneously and updates discount values correctly.					
TC_011	Validate shopping cart display order and Save for Later functionality	Medium/High	Cart contains multiple items	1. Add items at different times 2. Open cart 3. Select an item and click Save for Later	Most recently added item appears at the top. Selected item moves to Save for Later section successfully.					
TC_012	Validate Out of Stock Item behavior in shopping cart	Mandatory	Cart contains an item that becomes unavailable after adding	1. Open cart after item becomes unavailable 2. Attempt to update quantity	Red message 'Item went out of stock' appears below item and quantity update is disabled.					
TC_013	Validate order placement restriction with Out of Stock items	Mandatory	Cart contains at least one Out of Stock item	1. Proceed to checkout 2. Attempt to place order	User cannot place order until Out of Stock items are removed or replaced.					
TC_014	Validate shopping cart UI styling and cross-platform behavior	Low/Strict	Application available on Chrome Windows OS and iOS mobile app	1. Open cart in Chrome on Windows 2. Open cart in iOS mobile app 3. Verify layout, alignment, and functionality	Cart displays correctly with proper alignment and styling, and core functions work consistently across both platforms.					

Observation:

- Even though a new chat was created, it carried forward the user role, test case count, and other requirements to generate the current prompt response.
- Low and strict priority test cases were combined, even though they could have been split into individual test cases. This was done even though there was room for one more test case within the provided range (Given range: 12-15; generated count: 14).
- Beyond just the content, I can also design the format of the generated file. (For example, I requested a border/outline for the file, and it worked as expected).
- It did not consider the provided persona.

Scenario 3: API Testing

Weak Prompt: Write API test cases for user registration.

ChatGPT Response:

API Endpoint Example: POST /api/v1/users/register

Purpose: Validate user registration API functionality, input validation, security, and error handling.

Test Case ID	Test Scenario	Test Data / Request	Expected Result
API_REG_001	Verify user registration with valid details	Valid name, unique email, valid password, required fields	API should return 201 Created . User account should be created successfully.
API_REG_002	Verify registration with already registered email	Existing email address with valid user details	API should return 409 Conflict (or configured error code). Response

Test Case ID	Test Scenario	Test Data / Request	Expected Result
API_REG_003	Verify registration without mandatory fields	Missing email/password/name fields	should indicate email already exists. API should return 400 Bad Request with validation error details.
API_REG_004	Verify registration with invalid email format	"user@com" or "user.com"	API should reject request and return 400 Bad Request with email validation message.
API_REG_005	Verify registration with weak password	Password below minimum length or without required complexity	API should reject request with password policy validation error.
API_REG_006	Verify registration with password exceeding maximum length	Password length greater than allowed limit	API should return validation error indicating maximum length exceeded.
API_REG_007	Verify registration with empty/null values	Name/email/password sent as empty or null	API should return 400 Bad Request with appropriate field-level errors.
API_REG_008	Verify registration with special characters in input fields	Name containing unsupported characters or malicious input	API should validate input and prevent invalid data storage.
API_REG_009	Verify SQL injection prevention during registration	Email/name containing SQL payload (' OR 1=1 --)	API should safely reject/handle input without database impact.
API_REG_010	Verify XSS attack prevention during registration	Name field containing <script>alert(1)</script>	API should sanitize input and prevent script execution/storage.
API_REG_011	Verify duplicate registration request handling	Send same registration request multiple times	API should create only one account and handle duplicate requests gracefully.
API_REG_012	Verify response schema after successful registration	Valid registration request	Response should contain expected fields (user ID, message, timestamp) and should not expose sensitive data.
API_REG_013	Verify password is not returned in API response	Valid registration request	Response should not contain password or sensitive authentication details.
API_REG_014	Verify authentication token generation after registration (if applicable)	Successful registration	API should return valid access/verification token based on requirement.

Test Case ID	Test Scenario	Test Data / Request	Expected Result
API_REG_015	Verify registration API without authentication headers	Registration request without required headers (if applicable)	API should return appropriate authorization error.
API_REG_016	Verify unsupported HTTP methods	Send GET/PUT/DELETE request to registration endpoint	API should return 405 Method Not Allowed .
API_REG_017	Verify incorrect Content-Type header handling	Send request with text/plain instead of application/json	API should reject request with unsupported media type error.
API_REG_018	Verify registration with maximum allowed field lengths	Name/email/password at boundary length	API should successfully process valid boundary values.
API_REG_019	Verify registration API response time	Submit valid registration request under normal load	API response should be within defined SLA.
API_REG_020	Verify concurrent registration attempts with same email	Multiple simultaneous requests using same email	Only one user should be created; other requests should receive duplicate user error.

ICEPOT Prompt:

Context

Act as an API tester and write test cases to test the "User Registration" functionality of a Healthcare application. The component is to be validated through automation testing.

Instruction

[CRITICAL] Generate test cases to validate "User Registration" API through postman – JS programming language

[MANDATORY] Generate test cases to validate "User Registration" API through HTTP method as "POST"

[MANDATORY] Generate test cases to validate "User Registration" API with endpoint as v1/user/register

[HIGH PRIORITY] Generate test cases to validate "User Registration" endpoint with different environments (QA, STG)

[MANDATORY] Generate test cases to validate "User Registration" API with mandatory request header "**Key:** Content-Type ; **Value:** application/json"

[MANDATORY] Generate test cases to validate "User Registration" API requiring an API Key in the header to allow registration

[HIGH PRIORITY] Generate test cases to validate "User Registration" API response status code with a **400 Bad Request** if required fields are missing or improperly formatted (e.g., weak password, invalid email structure)

[HIGH PRIORITY] Generate test cases to validate "User Registration" API response status code with a **201 Created** and a JSON response containing confirmation details (e.g., a unique id, message, or generated token)

[HIGH PRIORITY] Generate test cases to validate "User Registration" API response status code with a **409 Conflict** or **400 Bad Request** status code if the username or email is already registered.

[LOW PRIORITY] Generate test cases to validate "User Registration" API by providing not properly formatted request (Eg: missing semicolons)

[MEDIUM PRIORITY] Generate test cases to validate "User Registration" API providing backend trims accidental leading or trailing spaces from text inputs instead of saving them into the database

Example:

Request Payload:

```
{
  "username": "testuser123",
  "email": "testuser123@example.com",
  "password": "SecurePassword123!",
  "fullName": "John Doe"
}
```

Test Scripts:

```
// Validate successful registration code
pm.test("Status code is 201", function () {
  pm.response.to.have.status(201);
});
```

```
// Verify response body returns an ID or success message
pm.test("Response contains user ID", function () {
  var jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property('id');
});
```

Success Response:

```
{
  "status": "success",
  "message": "User registered successfully.",
  "data": {
    "id": "usr_9f82b3c1a7d4e5f6",
    "username": "testuser123",
    "email": "testuser123@example.com",
    "fullName": "John Doe",
    "createdAt": "2026-06-13T22:46:00Z"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Persona

Act as an API Test lead to review the code

Output

Downloadable file supported by postman.

Tone:

Generate the code to be easily understandable

ChatGPT Response:

I created a **Postman Collection JSON file** for the Healthcare **User Registration API automation testing**.

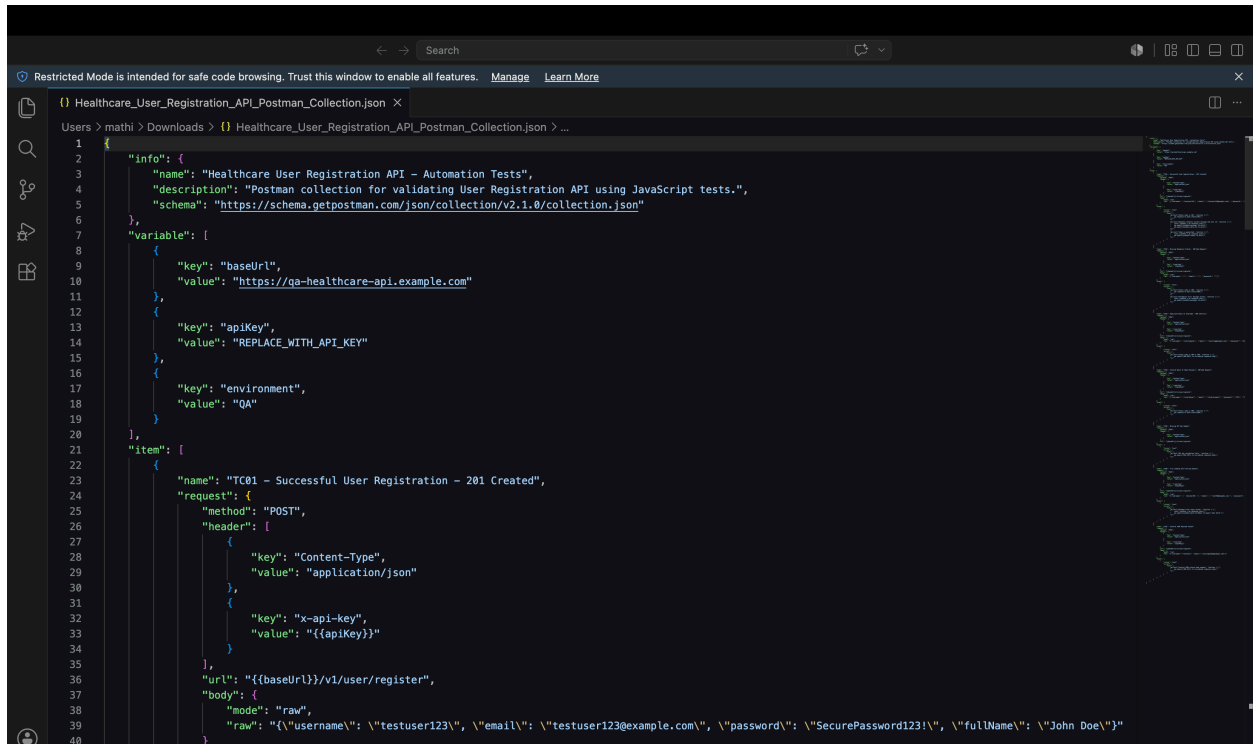
The collection includes:

- ☒ HTTP method validation: **POST**
- ☒ Endpoint validation: **v1/user/register**
- ☒ Environment variable support (**QA/STG via baseUrl variable**)
- ☒ Mandatory header validation:
 - Content-Type: application/json
 - API Key header (x-api-key)
- ☒ Positive test:
 - 201 Created
 - Response JSON validation
 - Unique user ID validation
 - Token validation
- ☒ Negative tests:
 - Missing mandatory fields → 400 Bad Request
 - Invalid email / weak password → 400 Bad Request
 - Duplicate username/email → 409 Conflict or 400 Bad Request
 - Missing API Key → 401/403
 - Invalid JSON payload format
- ☒ Input trimming validation using Postman JavaScript assertions

You can import the collection directly into Postman:

For execution:

1. Import the JSON file into Postman.
2. Create QA and STG environments.
3. Update:
 - `{{baseUrl}}`
 - `{{apiKey}}`
4. Run the collection using Collection Runner/Newman for automation execution.



```
1 {
2   "info": {
3     "name": "Healthcare User Registration API - Automation Tests",
4     "description": "Postman collection for validating User Registration API using JavaScript tests.",
5     "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
6   },
7   "variable": [
8     {
9       "key": "baseUrl",
10      "value": "https://qa-healthcare-api.example.com"
11    },
12    {
13      "key": "apiKey",
14      "value": "REPLACE_WITH_API_KEY"
15    },
16    {
17      "key": "environment",
18      "value": "QA"
19    }
20  ],
21  "item": [
22    {
23      "name": "TC01 - Successful User Registration - 201 Created",
24      "request": {
25        "method": "POST",
26        "header": [
27          {
28            "key": "Content-Type",
29            "value": "application/json"
30          },
31          {
32            "key": "x-api-key",
33            "value": "{{apiKey}}"
34          }
35        ],
36        "url": "{{baseUrl}}/v1/user/register",
37        "body": {
38          "mode": "raw",
39          "raw": "{\\\"username\\\": \\\"testuser123\\\", \\\"email\\\": \\\"testuser123@example.com\\\", \\\"password\\\": \\\"SecurePassword123!\\\", \\\"fullName\\\": \\\"John Doe\\\"}"
40        }
41      }
42    }
43  ]
44 }
```

Observation:

- Generated JSON file without explicitly mentioned
- Did additional tasks like creating baseUrl variable to support different environment execution
- Not sure about tracking the Priority provided
